

CS683 Project Assignment

iperf3 network tester
Jerold (Jerry) Abramson

1.	Overview	2
1.1.	More Details	2
1.2.	Motivation	2
2.	Related Work	3
3.	Requirement Analysis and Testing	4
4.	Design and Implementation	6
5.	Project Structure	9
6.	Timeline	9
7.	AI Usage Log	10
8.	Future Work (Optional)	10
9.	Project Demo Links	10

1. Overview

The application being developed will utilize the de-facto network protocol tool for testing network speeds between a client and a server.

Note: The application will only be implementing the client-side functionality of iPerf3, it is assumed that an iPerf3 server is running on another accessible via the Android client.

1.1. More Details

- For my project I am implementing a network performance monitor based on the iPerf3 protocol.
- Although there are a couple of Android apps that already partially implement this, one of them is terribly complex, quite expensive, (Analti), more focused on WiFi and internet performance monitoring.
- There is also a very old version that is nothing more than the text based version of the existing 'iperf3' application that runs in a console terminal window.
- There is a fairly nice version that runs on iOS, but that has not been ported to Android.

1.2. Motivation

- I utilize network testing tools as part of my "home-labb'ing" hobby.
- One of the tools I am constantly utilizing is based on the iPerf3 protocol, which is defined here:
- *For some context on iPerf3 is available → [here](#)*

1.3. Purpose

- Provide the ability to test WiFi signal strength using an Android phone.

2.Related Work

- I have developed a Java text-based console program that runs the iPerf3 native executable in the background, and provides performance details in a asynchronous manner.
- Here are a few samples:
-

Startup

```
newTestBandWidth.sh pve -R

[Sun Mar 15 21:33:48 EDT 2026] Executing iperf3 command-line:
/usr/bin/iperf3 -c pve -O 2 -P 8 -t 10 -R
Running: [      *]
```

Completion

```
[Sun Mar 15 21:33:48 EDT 2026] Executing iperf3 command-line:
==> '/opt/local/bin/iperf3' '--forceflush' '--connect-timeout' '3000' '-c'
'pve' '-O 2' '-P 8' '-t' '10' '-R' ''
Running: [      *]
Connecting to host pve, port 5201
Reverse mode, remote host pve is sending
Local Host/IP: 192.168.1.10 remote Host/IP: 192.168.1.7 Remote Port: 5201
Results:  0.00-10.00      *          ]      2.33 Gbits/sec  sender
          => 0.00-10.00      2.33 Gbits/sec  receiver

[Avg]          2.33 Gbits/sec
[Min]          1.16 Gbits/sec
[Max]          2.33 Gbits/sec

Return Code: 000 [Avg=2.33 Gbits/sec]
```

2.1. Similar Applications

App	Description	Play Store Link
Analti	<i>analiti Networking Experts</i>	link
iperf3	<i>Uncle Tools</i>	link

There is also a similar iOS application that I am using as a very basic model for my user interface.

However, this has not been ported to Android, and I also plan on making some adjustments based on my experience with iperf3.

3. Requirement Analysis and Testing

3.1. Requirement 1

Title	prototype of remote IP connection (ESSENTIAL)
Description	A hard-coded remote IP address will be used as the remote iPerf3 server
Acceptance Tests	runs a fixed loop of 10 iperf3 intervals over TCP/IP
Test Results	[Avg] 2.33 Gbits/sec
Mockup	Text based output —→ iperf3 -c 172.236.101.251: local 192.168.1.10 connected to 172.236.101.251 —→ Result: 940 Mbits/sec
Status	Iteration 1 - Perform a simple ping to remote host Iteration 2 (or possibly 3) - perform a single iperf3 test (count=1)

3.2. Requirement 2

Title	Client side host/ip address entry (ESSENTIAL)	
Description	The user shall be able to type in a host/ip address	
Mockups		
Acceptance Tests	host-name/IP Address validation	
Test Results	Positive Test: "google.com" Test runs Negative Test: "abcd.baddomain" <i>Validation error</i>	
Status	Iteration 1 - Allow entry of any text Iteration 2 - Validate remote host Iteration 3 - perform iperf3 test with hard-coded-count of 10	

3.3. Requirement 3

Title	Input desired count of iperf3 loops (<i>ESSENTIAL</i>)	
Description	The user shall be able enter a test count (i.e. 10 seconds)	
Mockups		
Acceptance Tests	Test count entry validation	
Test Results	<u>Postive Test</u> Enter 10 - test runs for 10 iterations <u>Negative Test</u> Enter -1 - input validation fails with error message	
Status	Iteration 1 - User Interface allows manual keyboard entry of count Iteration 4 - iperf3 runs for specified count against specified host	

3.4. Requirement 4

Title	Unit of measurement
Description	The user shall be able to specify the unit of measurement for output bandwidths
Mockups	Entry of MB, Entry of gb, Entry of bytes
Acceptance Tests	Validate rationale unit text
Test Results	Positive: MB, Negative foobar
Status	Iteration 1 - Allow keyboard entry of unit Iteration 2 - validate unit of measurement Normalize output results to specified unit

4.

4.Design and Implementation

I utilized the new <https://claude.ai/> capability for render of diagrams without the need for Canvas, PlantUML, or other tools.

I plan on making significant changes to this initial proposal, but I truly believe that this is a really good starting point.

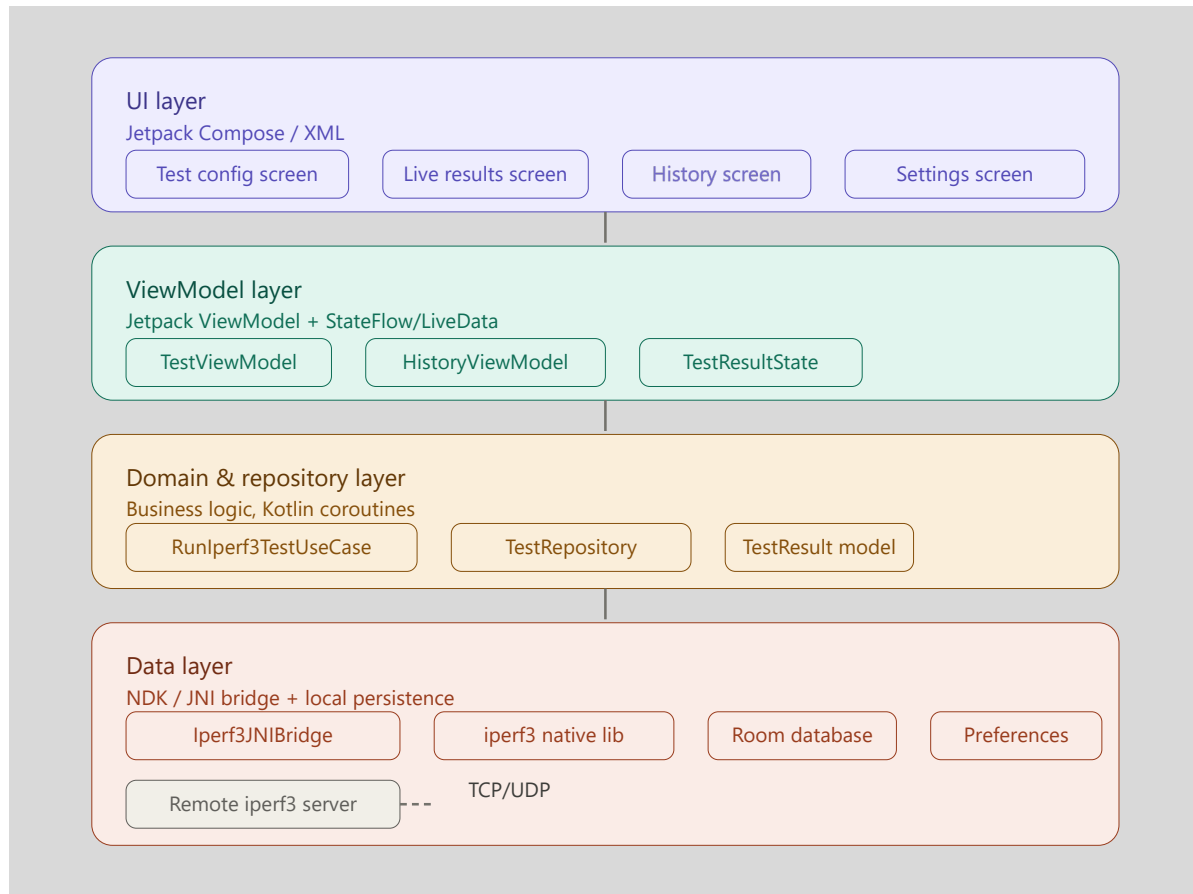
I am super impressed with what the AI could do, now that it is able to output diagrams.

I will be using the Microsoft Word Track changes capabilities as I modify the provided AI design and architecture.

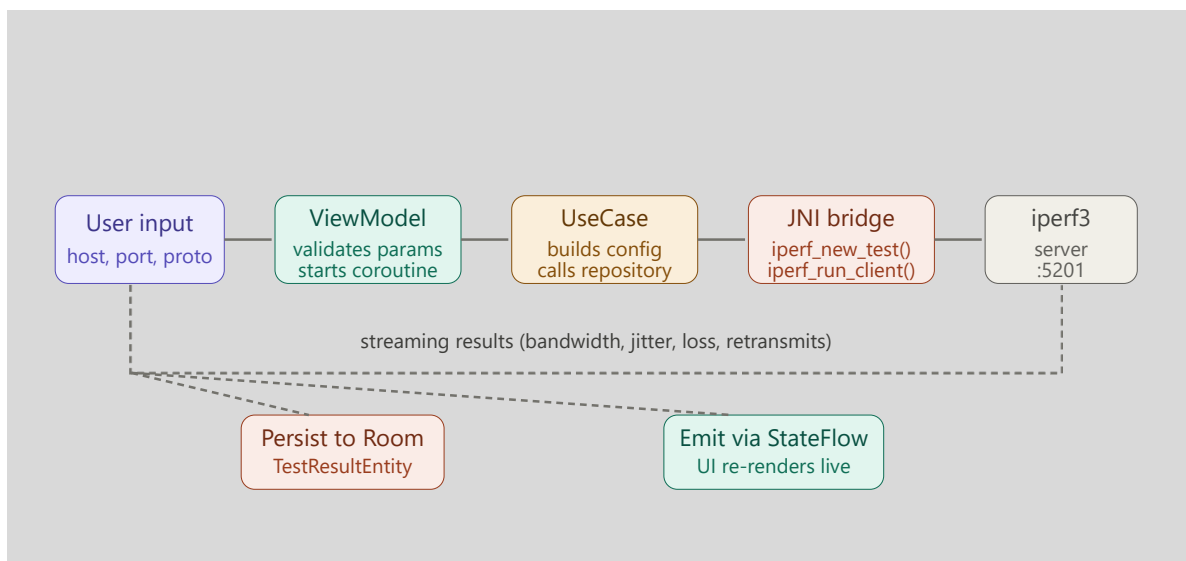
AI Prompt:

Design the basic architecture for an Android application that performs iperf3 tests against a remote iperf3 server

4.1. Layered Architecture



4.2. Test Execution Flow



4.3. Full component breakdown organized by prose

Here's a breakdown of the architecture, organized across three views: the layered app architecture, the data flow for a test run, and the component detail.

Layered architecture — the app is organized into four tiers from UI down to the native `iperf3` binary:
Test execution flow — what happens when the user taps "Run test":---

Here's the full component breakdown in prose, organized by layer:

UI layer (Jetpack Compose) — four screens: a test configuration screen (host, port, protocol, duration, parallel streams, direction), a live results screen with a real-time chart of throughput, a history screen backed by a `RecyclerView` or `LazyColumn`, and a settings screen for saved server profiles.

ViewModel layer — `TestViewModel` owns the coroutine scope and exposes a `StateFlow<TestUiState>` that the UI collects. States cycle through `Idle` → `Connecting` → `Running(progress)` → `Complete(result)` → `Error(reason)`. `HistoryViewModel` paginates results from Room via a `Paging 3` data source.

Domain layer — `RunIperf3TestUseCase` accepts a `TestConfig` data class (host, port, protocol, duration, parallel, reverse) and returns a `Flow<TestEvent>` so intermediate interval results stream up while the test runs. A `TestRepository` interface abstracts both the JNI bridge and the local database so the domain layer stays decoupled from Android specifics.

Data layer — the critical piece — `iperf3` is a C library, so it's compiled as a shared object (`.so`) via the Android NDK and bundled in `src/main/jniLibs/`. A thin JNI wrapper (`Iperf3Bridge.kt` + `iperf3_bridge.cpp`) translates Kotlin calls into `iperf_new_test()`, `iperf_set_*`, and `iperf_run_client()` calls from `libiperf.h`. Interval callbacks from the C library are marshalled back to Kotlin via a JNI callback interface and posted onto a `Channel<TestEvent>`. Room stores `TestResultEntity` rows with columns for timestamp, host, protocol, bandwidth (bps), jitter (ms), packet loss (%), and retransmits. A `DataStore Preferences` instance persists default server profiles and app settings.

Key design decisions to make early:

The biggest one is whether to use `libiperf` directly via JNI (cleaner, streaming results, no process overhead) or shell out to the `iperf3` binary via `ProcessBuilder` (much easier to implement but requires bundling the binary and dealing with stdout parsing). The JNI approach is the right one for a production app.

For networking, `iperf3` opens its own socket to port 5201 (configurable) on the server — the app just needs `INTERNET` permission. You'll also want `ACCESS_NETWORK_STATE` to gate tests on connectivity and to tag sockets with `TrafficStats` for per-UID accounting.

Concurrency should run entirely on `Dispatchers.IO` inside a `viewModelScope` coroutine, with the native blocking `iperf_run_client()` call wrapped in `withContext(Dispatchers.IO)` to avoid blocking the main thread. Cancel support comes from a `Job` that, when cancelled, calls `iperf_reset()` on the native side.

5. Project Structure

This is not needed for iteration 0)

6. Timeline

Iteration	Application Requirements • Essential • Desirable • Optional	Android Components and Features to be used
0.5	Layout Palette Utilized to design UI skeleton <code>activity_main.xml</code>	Layout editor
1	Essential: Simple ping to hard-coded remote host	TBD (Networking)
1	Essential: Allow entry of simple hostname (no validation or usage in iteration 1)	TextView
1	Essential: Manual keyboard entry of test count	TextView
1		
2 or 3	Essential: perform a single <code>iperf3</code> test (count=1)	TBD (Consuming <code>iperf3</code> native executable)
2	Desirable: Validate remote host	Networking
3	Essential: Perform <code>iperf3</code> to specified host with a count of 1	TBD
4	Essential: <code>iperf3</code> runs against specified host for specified count with specified unit	TBD

7.AI Usage Log

Tool	Task	Evaluation	Links or prompt history
1	https://claude.ai	Positive	Claude AI Public Link

- Future Work (Optional)

This section is optional, and you can include this section in the final iteration if you want.)

- Project Demo Links

*(For on campus students, we will have project presentations in class. For online students, you are required to submit a video of your project presentation which includes a demo of your app and explanation of your implementation. You can use Kaltura or zoom or any video tool to make the video and then submit it on blackboard. Please check the following link for the details of using Kaltura to make and submit videos on blackboard. You can also use other video tools and upload your video to youtube if you like:
https://onlinecampus.bu.edu/bbcswebdav/courses/00cwr_odeelements/metcs/cs_Kaltura.htm)*

- References

(any references you used for the project)